

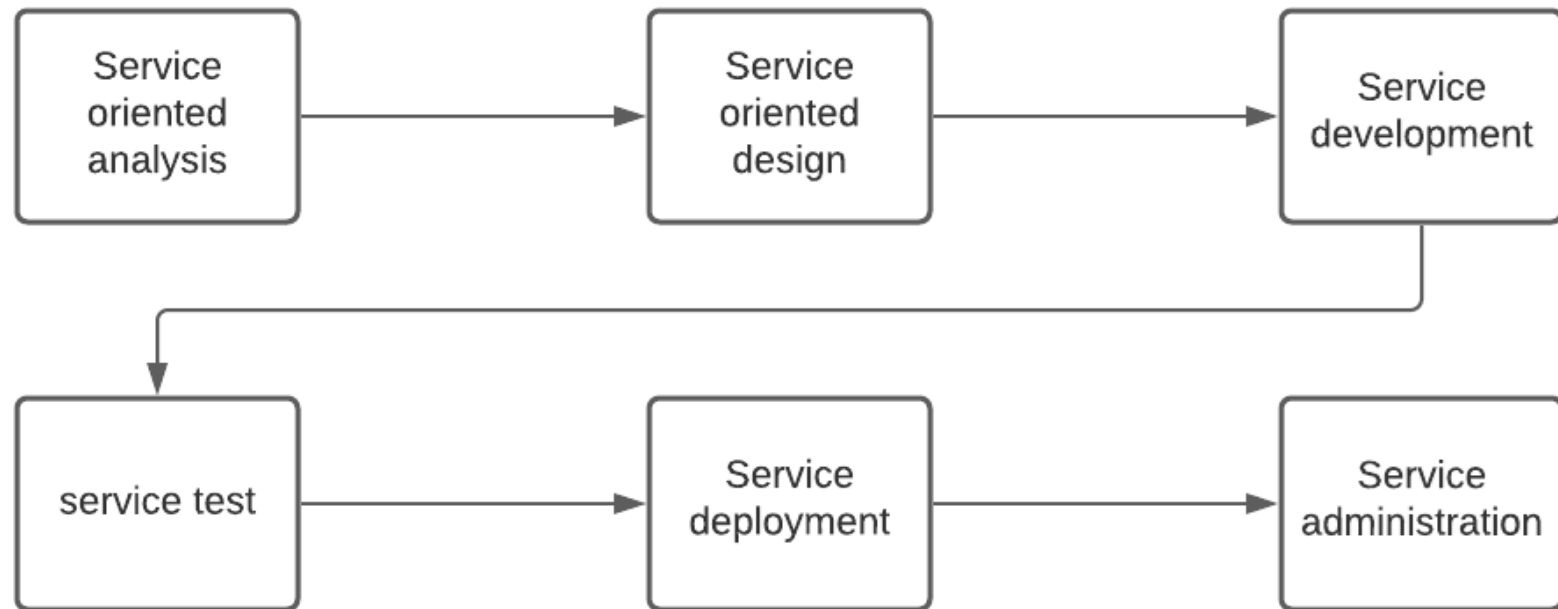
CS383 - Software Engineering

Software Architecture (II)

Service-oriented architecture (SOA)

- A way of developing distributed systems in which components are web services
 - Mostly used for SaaS applications
- Services are executing on geographically distributed computers
- Just like any software, a service application have lifecycle

SOA delivery lifecycle



SOA analysis



- Establish an understanding about the system in question
 - New system: identify stakeholder requirements
 - Existing system: understand functionality, infrastructure
- Identify the scope of services
 - E.g. what set of functionalities should be made as available services
- Identify likely clients of the services
- Identify performance/availability
- Identity candidate service operations and relevant data structure
 - identify candidates input, output, potential faults (informally)

SOA design



- Refine candidates into formal service operations
- Selected principles of design:
 - Statelessness: ideally, each request should be independent transaction
 - Abstraction: internal implementation details shouldn't be exposed to clients.
 - Reusability/composability: duplicate behaviour should be organized into composable service operations
 - You can consider other principle design (process attributes)

SOA testing

- Test services under normal/peak/overload conditions
 - Normal: good service performance
 - Peak: acceptable service performance
 - Overload: services remains operational (do not crash)
- Performance
 - Response time
 - Throughput (e.g. requests per second)
 - Resource usage (e.g. CPU, memory)

SOA deployment

- Identify issues not seen in standalone systems
 - Distribution and replication of services
 - E.g. Fail-over, load balancing behaviour
 - Co-existence of multiple versions of the same service
 - E.g support clients who have different version that the latest interface
- Run performance tests on deployed services

SOA administration

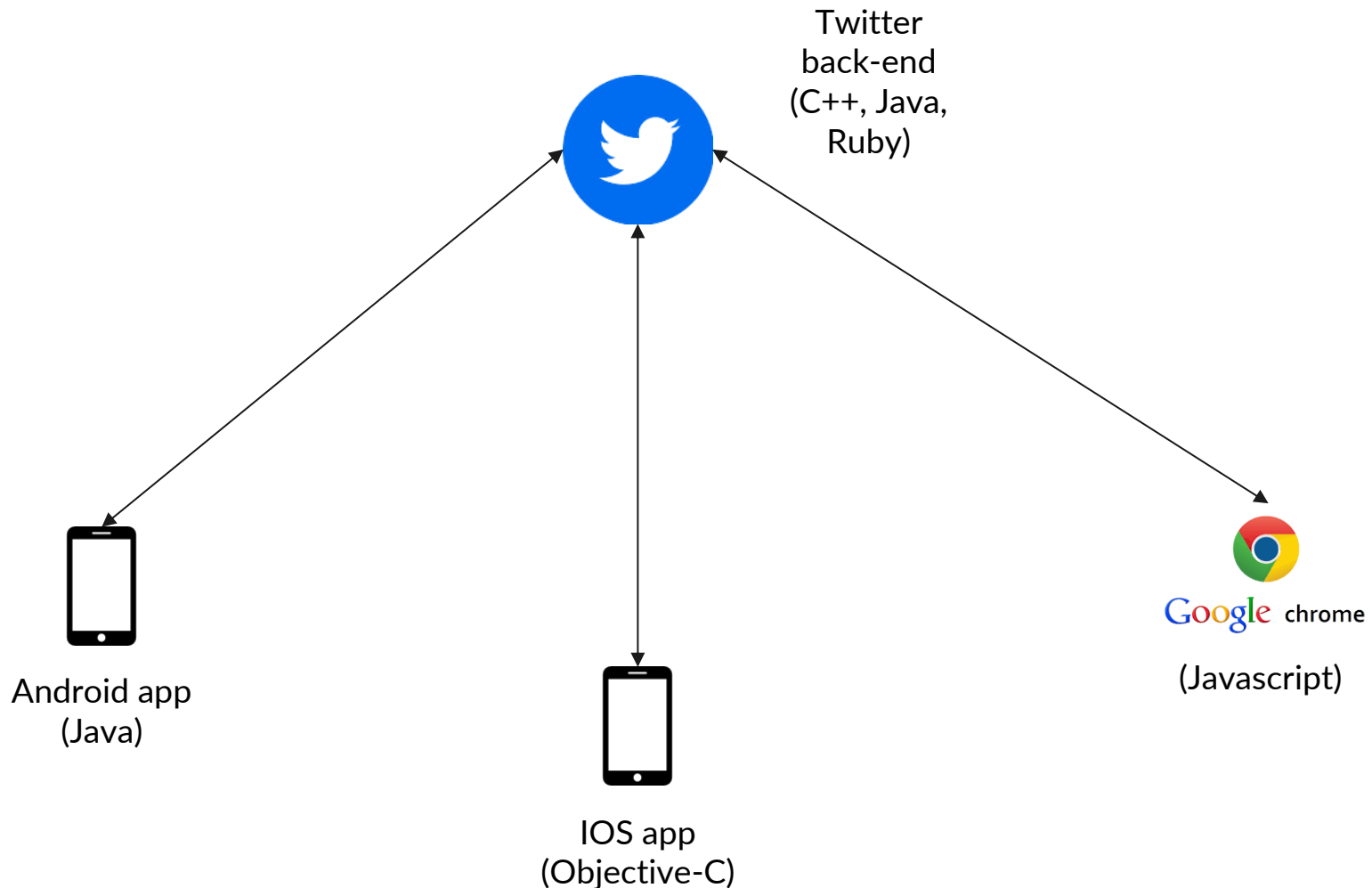
- Performance monitoring
- Dynamic resource allocation
- Bottleneck detection: the capacity of the system under monitoring is limited to a lowest throughput possible

Heterogeneity and distribution applications



- Heterogeneous: application are different in terms of
 - Implementation language
 - Storage mechanisms and format
 - Operating systems and hardware
- Distribution: application can be across
 - Different machines
 - Physical locations
- However, these applications need to interoperate with each other

Heterogeneity and distribution applications



Primitive Integration Patterns



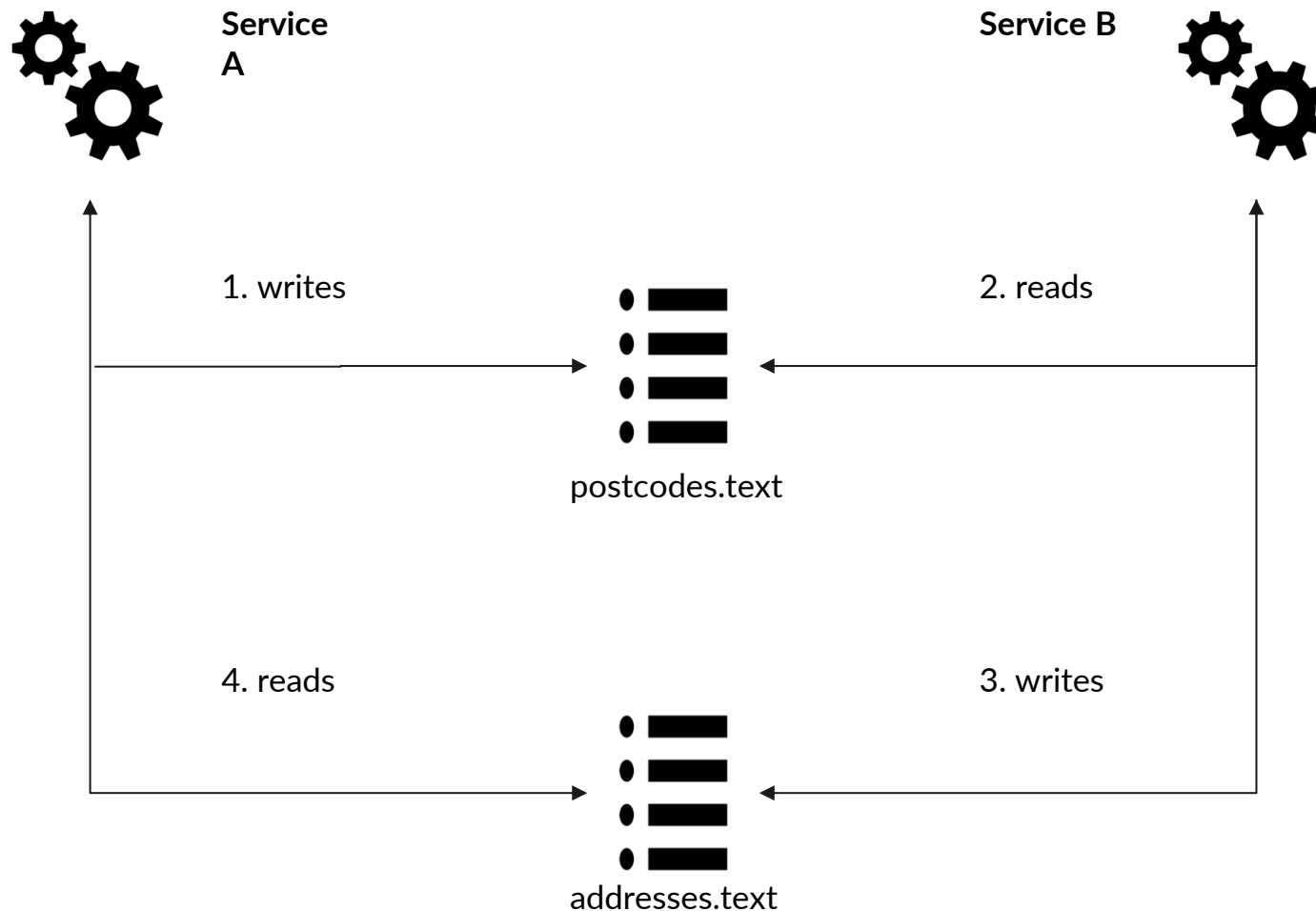
- Describes the ways of communication and information exchange between systems.
- There are three primitive (common) ways of integration:
 - File-based communication
 - Common database
 - Sockets

Communication/Integration scenario



- Service A and B can communicate
- Service A wants to resolve postcodes and therefore communicate with Service B
- Service B takes any postcode given by Service A and creates entries of addresses and makes it available to Service A
- Service A reads the information put by Service B and use it.

File-based communication

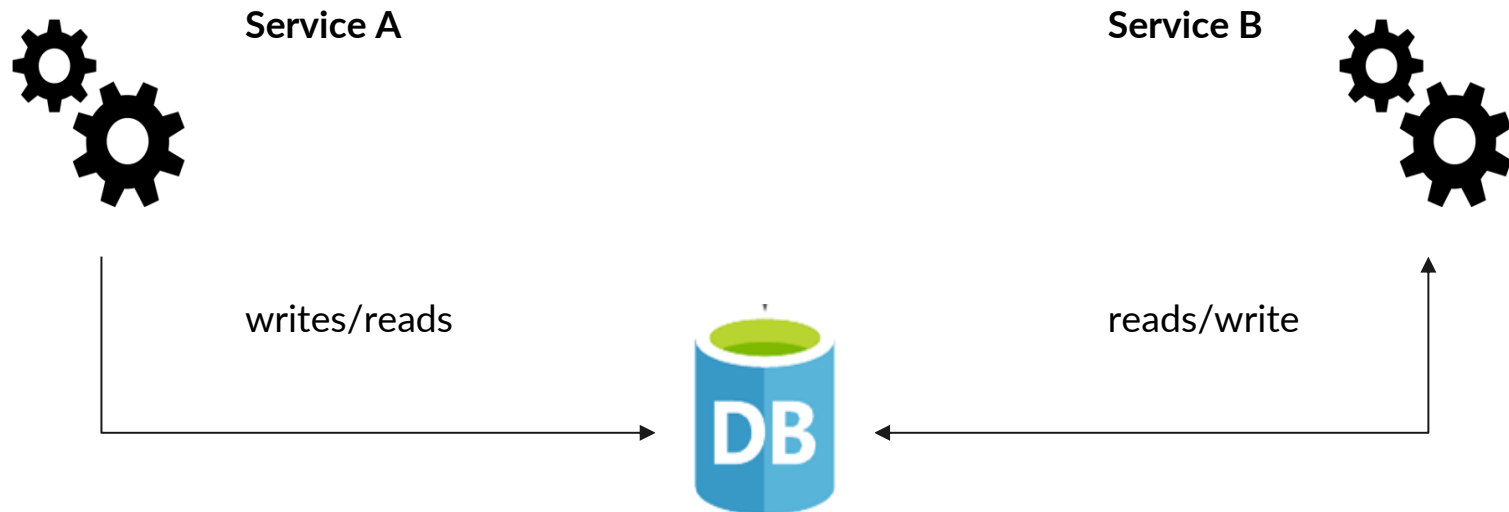


File-based communication

- Not real-time
 - Lag between A writing/reading and B reading/writing
- File locking problems
 - Only one action is allowed in a file
- Ad-hoc file formats
 - A and B must agree upon a file format to use
- Hard to scale for multiple clients
- Housekeeping
 - Clean files after they are big, no need for them anymore, or corrupted

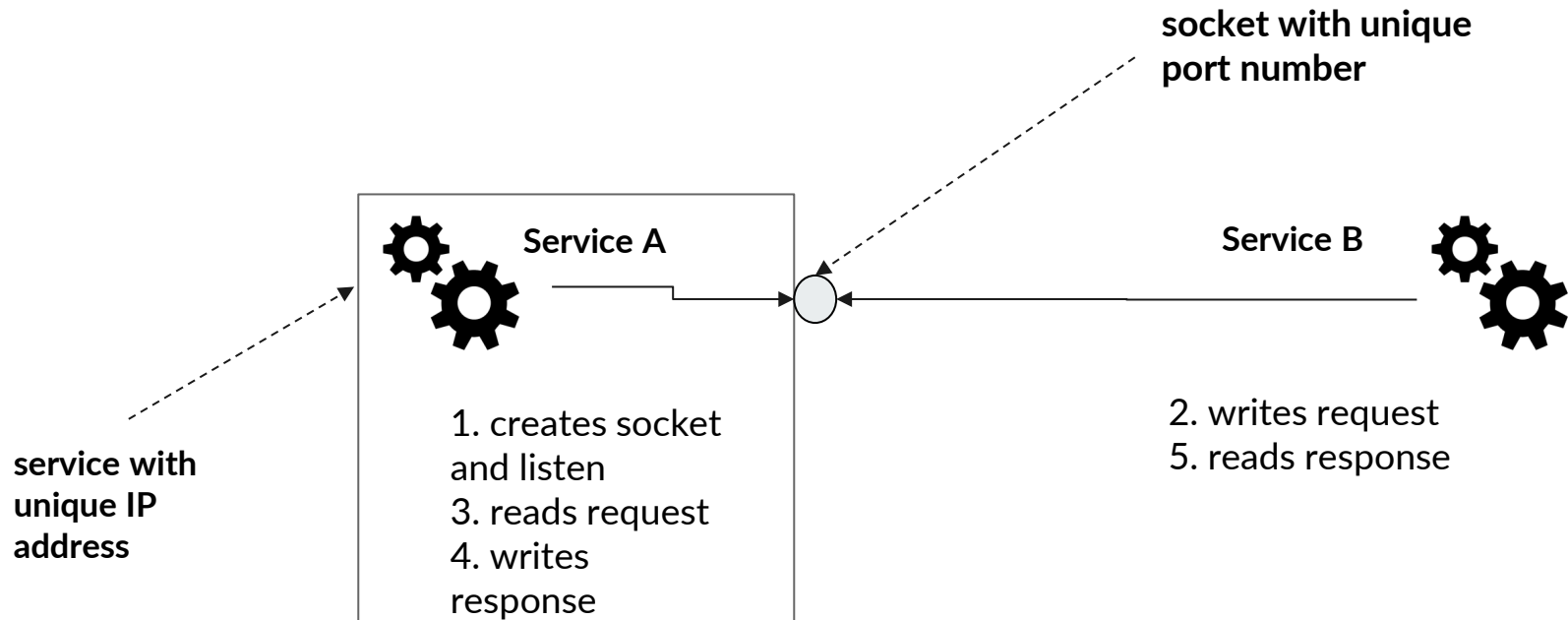
Common database

- Advantages:
 - Access synchronization
 - Ad-hoc file formats
 - Multiple clients can access
- Still have problems:
 - Housekeeping

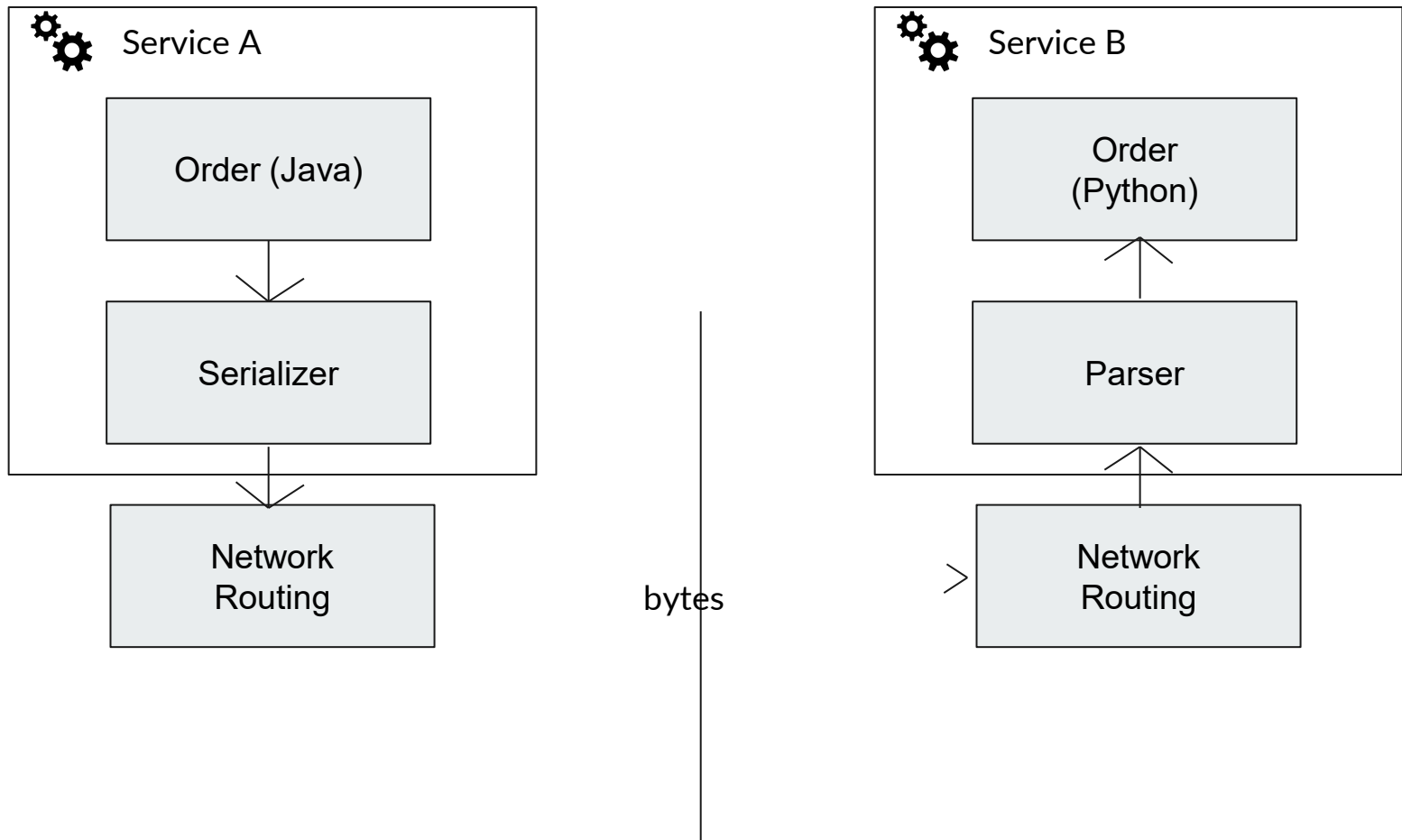


Sockets

- A communication endpoint that be uniquely named and addressed in network
- A service provide can open and listen to socket at a port number
- A client can connect to opened socket using IP address and port
- Both the provider and client can read/write from/to the socket



Data exchange format



Data exchange format (cont.)



- Serialization is a set of rules that defines how bytes/characters should be interpreted
 - E.g, an order has orderId and then sequence of product information as [productId -> quantity] combined with a bar
 - E.g, o1731: p313->4 | p426->1 | ...
- The set of rules of serialization and parsing must be known by all parties

Data exchange format (cont.)

- Writing parsers/serializers from scratch is not ideal task
 - Time consuming
 - Error-prone
 - Low-level
- Any changes to B's parsing, must be addressed in A's serializer

eXtensible Markup Language (XML)



- Support for different structured data
- Platform and machine independent
- Text based
- Machine processable
- Extensible

eXtensible Markup Language (XML)



```
<library> <!-- begin a tag library -->
  <book title="Software Engineering" pages="790"> <!-- begin a tag book with properties -->
    <author>Ian Sommerville</author> <!--a relationship from book to an author -->
    <publisher>Addison-Wesley</publisher> <!--a relationship to a publisher -->
    <year>2011</year> <!--a relationship to a year -->
  </book> <!-- end of tag book -->
  .... <!-- many other book instances and tags -->
  ....
</library> <!-- end of tag library -->
```

eXtensible Markup Language (XML)

